

Tema 5. Polimorfismo

1. Brevemente, ¿qué es el **"polimorfismo"** y para qué sirve en programación orientada a objetos? ¿qué es la **"sobreescritura"** de métodos?

Respuesta

El polimorfismo es la capacidad de tratar objetos de distintos tipos de forma uniforme mediante una referencia común, normalmente del tipo de la superclase. Permite que un mismo método se comporte de manera diferente según el objeto real sobre el que se invoca. Esto facilita escribir código más general, reutilizable y extensible.

La sobreescritura (overriding) ocurre cuando una subclase redefine un método heredado de la superclase, manteniendo la misma firma (nombre y parámetros), pero cambiando su comportamiento. Así, aunque se invoque el método desde una referencia del tipo base, se ejecuta la versión específica del objeto real.

2. ¿En qué consiste la **"ligadura dinámica"** o **"enlace tardío"**? ¿qué relación tiene con el polimorfismo? ¿hay que indicarlos explícitamente al programar o depende esto del lenguaje? Compara C++ y Java. Indícalo después también para Python.

Respuesta

La ligadura dinámica (o enlace tardío) es el mecanismo por el cual la decisión de qué método ejecutar se toma en tiempo de ejecución, en función del tipo real del objeto, y no del tipo de la referencia. Esto es lo que permite el polimorfismo en lenguajes orientados a objetos.

En Java, la ligadura dinámica ocurre automáticamente para métodos no estáticos, por lo que no es necesario indicarla explícitamente. En cambio, en C++ hay que usar la palabra clave virtual para que el comportamiento sea dinámico; si no, se usa enlace estático.

En Python, todo es dinámico por defecto. No se necesita ninguna palabra clave para activar el polimorfismo, ya que el lenguaje decide en tiempo de ejecución qué método ejecutar según el objeto.

3. Pon un ejemplo sencillo en Java, de un **Soldado**, con un método **saluda**, con dos subclases: **Zapador** y **Artillero**, donde **Zapador** sobreescribe el método **saludar**, sustituyendo por completo su comportamiento. Ilustra el funcionamiento del polimorfismo creando un array de **Soldados** de dos tipos y luego recorriéndolo empleando referencias de tipo **Soldado** y llamando a **saludar**.

Respuesta

Se puede ilustrar el polimorfismo creando una clase base Soldado con un método saludar, y subclases que lo sobreescriben. Aunque se usen referencias de tipo Soldado, el comportamiento dependerá del tipo real del objeto.

```
class Soldado { public void saludar() { System.out.println("Soy un soldado"); } }

class Artillero extends Soldado { @Override public void saludar() { System.out.println("Soy un artillero"); } }

class Zapador extends Soldado { @Override public void saludar() { System.out.println("Soy un zapador"); } }

public class Main { public static void main(String[] args) { Soldado[] ejercito = new Soldado[2]; ejercito[0] =
new Artillero(); ejercito[1] = new Zapador();

    for (Soldado s : ejercito) {
        s.saludar();
    }
}
```

Aunque el array es de tipo Soldado, cada objeto ejecuta su propia versión del método.

4. Si sobrescribo un método, ¿puedo invocar el método base para trabajar a partir de su resultado? Haz que zapador cambie ligeramente la forma de saludar, que salude de forma normal, tal cual hace el soldado base, pero que además añada un "ZAPADOR A SUS ORDENES" ¿qué palabra clave del lenguaje has usado para invocar al método de la clase base?

Respuesta

Sí, al sobrescribir un método se puede invocar el método de la superclase para reutilizar parte de su comportamiento. Esto es útil cuando se quiere extender el comportamiento original en lugar de reemplazarlo completamente.

Para ello se utiliza la palabra clave `super`, que permite acceder a los métodos de la clase base. Así, primero se ejecuta el comportamiento original y luego se añade lógica adicional.

```
class Zapador extends Soldado { @Override public void saludar() { super.saludar();
System.out.println("ZAPADOR A SUS ORDENES"); } }
```

De este modo, el zapador saluda como un soldado normal, pero añade su mensaje específico.

5. Al sobrescribir un método en Java, ¿qué restricciones existen sobre los tipos de los parámetros y el tipo de retorno? ¿Qué diferencia hay entre sobrescritura (*overriding*) y sobrecarga (*overloading*)? ¿Para qué sirve la anotación `@Override` y por qué es recomendable usarla siempre?

Respuesta

Al sobrescribir un método en Java, los parámetros deben ser exactamente iguales a los del método original. El tipo de retorno debe ser el mismo o uno compatible (covariante). No se pueden cambiar los tipos de los parámetros, ya que entonces sería otro método distinto.

La diferencia entre overriding y overloading es que el primero redefine un método heredado, mientras que el segundo consiste en tener varios métodos con el mismo nombre pero diferentes parámetros dentro de la misma clase.

La anotación @Override indica que se está sobrescribiendo un método de la superclase. Es recomendable usarla siempre porque el compilador puede detectar errores si la firma no coincide exactamente.

6. Entonces, cuando se estudia Java, ¿se emplea el polimorfismo desde el principio? Por ejemplo, sobrescribiendo `toString` o sobrescribiendo `equals`, ¿ya estoy usando polimorfismo?

Respuesta

Sí, el polimorfismo se empieza a usar desde el principio al trabajar con Java, incluso sin ser consciente de ello. Por ejemplo, al sobrescribir métodos como `toString` o `equals`, ya se está utilizando polimorfismo.

Esto ocurre porque esos métodos están definidos en la clase `Object`, y cada clase puede proporcionar su propia implementación. Cuando se llama a `toString`, Java decide en tiempo de ejecución qué versión ejecutar.

Por tanto, incluso en ejemplos básicos, el polimorfismo ya está presente como parte del funcionamiento del lenguaje.

7. ¿Qué es una "**clase abstracta**"? ¿Qué es un "**método abstracto**"? ¿Puedo crear instancias de una clase abstracta? Pongamos un ejemplo en Java: Redefinamos `Soldado`, hagamos que, además del método `saluda` que ya tenía, tenga un método `atacar`, que sea abstracto y que cada tipo de soldado haga su acción cuando se le pida atacar. ¿Donde debemos poner `abstract`?

Respuesta

Una clase abstracta es una clase que no puede instanciarse directamente y que sirve como base para otras clases. Puede contener métodos normales y también métodos abstractos.

Un método abstracto es un método que no tiene implementación y que debe ser implementado por las subclases. Se declara usando la palabra clave `abstract`.

```
abstract class Soldado { public void saludar() { System.out.println("Soy un soldado"); }
```

```
    public abstract void atacar();
```

```
}
```

```
class Artillero extends Soldado { @Override public void atacar() { System.out.println("Disparando cohetes"); } }
```

```
class Zapador extends Soldado { @Override public void atacar() { System.out.println("Colocando minas"); } }
```

No se pueden crear objetos de `Soldado`, pero sí de sus subclases, que implementan el método `atacar`.

8. ¿Qué efecto tiene la palabra clave **final** sobre métodos y clases en Java? ¿Cómo se relaciona con el polimorfismo? ¿Conoces algún ejemplo de clase **final** en la propia API estándar de Java?

Respuesta

La palabra clave final impide la modificación de algo. Si se aplica a una clase, impide que se herede de ella. Si se aplica a un método, impide que sea sobrescrito.

Esto limita el polimorfismo, ya que evita que se cambie el comportamiento en subclases. Por tanto, usar final reduce la flexibilidad pero puede aumentar la seguridad del diseño.

Un ejemplo clásico en la API de Java es la clase String, que es final, lo que significa que no se puede heredar de ella.

9. En Java, qué son las **"interfaces"**? ¿Son como clases abstractas? ¿Una clase puede implementar más de una interfaz?

Respuesta

Las interfaces son contratos que definen métodos sin implementación (aunque en versiones modernas pueden tener implementaciones por defecto). Una clase que implementa una interfaz se compromete a implementar esos métodos.

Son similares a clases abstractas, pero no tienen estado (atributos de instancia) como las clases normales. Se usan para definir comportamientos comunes.

En Java, una clase puede implementar varias interfaces, lo que permite una forma de herencia múltiple controlada sin los problemas de la herencia múltiple de clases.

10. Vamos a poner un ejemplo nuevo con polimorfismo. Queremos implementar una clase **Punto**, con un método **calcularDistanciaA**, que permite calcular la distancia a otro **Punto**. Sin embargo, como queremos trabajar con puntos 2D y 3D, haz que ese método sea abstracto y haya dos implementaciones de ese cálculo de distancia. Emplea **instanceof** y *downcasting* para verificar que se recibe un punto compatible y poder calcular correctamente la distancia siempre entre puntos del mismo subtipo. Aprovecha este diseño para crear ahora una clase **Linea**, que acepta **Punto**, sin saber de qué tipo es, y es capaz de dar su longitud independientemente de las dimensiones de sus puntos (las cuales desconoce).

Respuesta

Se puede diseñar una clase abstracta Punto con un método abstracto para calcular distancias. Luego se crean subclases para 2D y 3D que implementan ese cálculo.

```
abstract class Punto { public abstract double calcularDistanciaA(Punto otro); } class Punto2D extends Punto { private final double x, y;
```

```
public Punto2D(double x, double y) {
    this.x = x;
    this.y = y;
}

@Override
public double calcularDistanciaA(Punto otro) {
    if (!(otro instanceof Punto2D)) {
        throw new IllegalArgumentException("Tipos incompatibles");
    }

    Punto2D p = (Punto2D) otro;
    double dx = p.x - this.x;
    double dy = p.y - this.y;
    return Math.sqrt(dx * dx + dy * dy);
}
```

```
} class Punto3D extends Punto { private final double x, y, z;
```

```
public Punto3D(double x, double y, double z) {
    this.x = x;
    this.y = y;
    this.z = z;
}

@Override
public double calcularDistanciaA(Punto otro) {
    if (!(otro instanceof Punto3D)) {
        throw new IllegalArgumentException("Tipos incompatibles");
    }

    Punto3D p = (Punto3D) otro;
    double dx = p.x - this.x;
    double dy = p.y - this.y;
    double dz = p.z - this.z;
    return Math.sqrt(dx * dx + dy * dy + dz * dz);
}
```

```
} class Linea { private final Punto p1, p2;
```

```
public Linea(Punto p1, Punto p2) {
    this.p1 = p1;
    this.p2 = p2;
}

public double longitud() {
```

```
        return p1.calcularDistanciaA(p2);  
    }  
  
}
```

La clase Linea no necesita saber si trabaja con puntos 2D o 3D. El polimorfismo permite que el cálculo correcto se haga automáticamente.

11. ¿Qué es la **"herencia de interfaces"** en Java? ¿Existe **"herencia múltiple de interfaces"**? Pon un ejemplo de una interfaz **Fichero** que tenga un método para leer su contenido en forma de **String** y luego dicha interfaz sea extendida por otra que sea **FicheroEscribible** que permita enviar contenido e incluso eliminar el fichero.

Respuesta

La herencia de interfaces consiste en que una interfaz puede extender otra interfaz, heredando sus métodos. Esto permite construir jerarquías de comportamiento.

En Java sí existe herencia múltiple de interfaces. Una interfaz puede extender varias interfaces, y una clase puede implementar varias interfaces.

```
interface Fichero { String leer(); }  
interface FicheroEscribible extends Fichero { void escribir(String contenido);  
void eliminar(); }
```

Una clase que implemente FicheroEscribible debe implementar todos los métodos de ambas interfaces.